Code Coverage Analysis

PES1UG24CS490 | Lab 7: Fuzz Testing for Input Validation Bugs

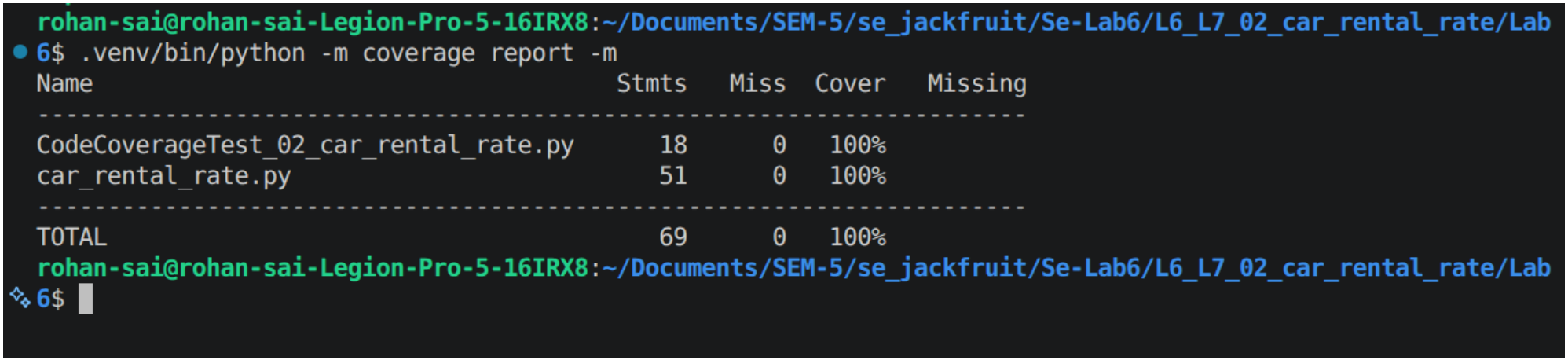


Figure 1. Code Coverage Analysis

Observation

The screenshot reports 100% line coverage for car_rental_rate.py: all 51 executable statements were covered, with no missed statements. Including CodeCoverageTest_02_car_rental_rate.py, the total is 69 statements with no misses. This screenshot measures the car-rental module shown in the report; the processor.py fuzz-test results are presented separately below.

Branch Coverage Analysis


```
rohan-sai@rohan-sai-Legion-Pro-5-16IRX8:~/Documents/SEM-5/se_jackfruit/Se-Lab6/L6_L7_02_car_rental_rate/Lab
6$ .venv/bin/python -m coverage report -m
Name                                      Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------------
BranchCoverageTest_02_car_rental_rate.py      33      0      0      0   100%
car_rental_rate.py                            51      0     36      0   100%
-------------------------------------------------------------------------------------
TOTAL                                         84      0     36      0   100%
rohan-sai@rohan-sai-Legion-Pro-5-16IRX8:~/Documents/SEM-5/se_jackfruit/Se-Lab6/L6_L7_02_car_rental_rate/Lab
6$
```

Figure 2. Branch Coverage Analysis

Observation

The screenshot reports 100% coverage for car_rental_rate.py, with 51 statements, 36 branches and no partial branches. Including BranchCoverageTest_02_car_rental_rate.py, the total is 84 statements. All measured branches of the displayed car-rental module were exercised by this test run.

Lab 7 — Fixed Application Output (Step 5)


```
rohan-sai@rohan-sai-Legion-Pro-5-16IRX8:~/Documents/SEM-5/se_jackfruit/Se-Lab6/L6_L7_02_car_rental_rate/Lab
7$ .venv/bin/python processor.py
==== Running sanitize_license_plate ====
Enter license plate with delimiters (-, _, #): KA-01-AB_1234
Sanitized Plate: KA01AB1234

==== Running parse_trip_mileage ====
Enter CSV of mileages (e.g. 100.5, 45.2, 80.0): 100.5, 45.2, invalid, , 80.0
Parsed Mileage List: [100.5, 45.2, 80.0]

==== Running format_rental_invoice ====
Enter rental record (Driver:Days:Category): Rohan:3:SUV
Rental Invoice: Driver Rohan booked SUV for 3 days: Total $150.00
rohan-sai@rohan-sai-Legion-Pro-5-16IRX8:~/Documents/SEM-5/se_jackfruit/Se-Lab6/L6_L7_02_car_rental_rate/Lab
7$
```

Figure 3. Lab 7 — Fixed Application Output (Step 5)

Observation

The plate KA-01-AB_1234 is normalized to KA01AB1234. The mileage parser keeps 100.5, 45.2 and 80.0 while ignoring the invalid token and blank field. The rental record Rohan:3:SUV produces a $150.00 invoice, consistent with three days at $50.00 per day. This run demonstrates the fixed application handling the supplied malformed mileage input without crashing.

Lab 7 — Passing Hypothesis Tests (Step 6)

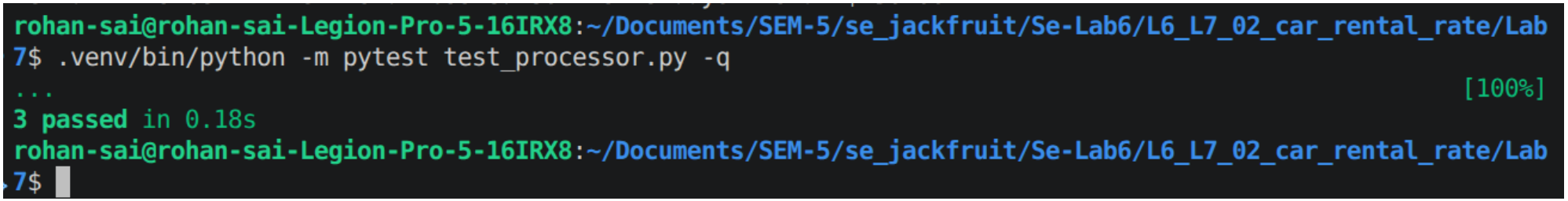


Figure 4. Lab 7 — Passing Hypothesis Tests (Step 6)

Observation

Pytest reports 3 passed in 0.18 seconds. The three Hypothesis test functions exercise license-plate sanitization, mileage parsing and rental-invoice formatting with generated text and None inputs. The number 3 refers to test functions, not the number of generated examples. Passing these tests supports robustness for the tested inputs; it does not prove all possible inputs are correct.

Lab 7 deliverables check

The fixed application (processor.py) and completed tests (test_processor.py) are provided in this Lab 7 folder. The supplied screenshots cover Steps 5 and 6. The handout also requests the buggy application output (Step 2) and failing fuzz tests (Step 4); those two screenshots were not supplied and are still required for complete before-and-after evidence.